

Backend Progress Update

Phase 1 Foundation | Day 2 Complete

We have moved from a basic placeholder server to a production-ready API foundation built on NestJS (Node.js + TypeScript). This covers infrastructure: API structure, database wiring, environment setup, and monitoring endpoints. Business features (products, orders, CRM, payments) are next.

What's Done

- API architecture: modular NestJS backend (Config, Database, Health modules)
- PostgreSQL connection layer for local dev (Docker) and production (Railway/Supabase)
- Environment config with separate dev and production templates
- Live endpoints tested: GET /api/health and GET /api/version
- Deployment prep: Vercel serverless handler + Docker Compose for local Postgres

API Endpoints (Live)

GET /api/health	Server status + database check
GET /api/version	API version, environment, build info

Tech Stack

NestJS 11 + TypeScript | PostgreSQL 16 | @nestjs/config | Docker Compose (local) | Planned: Railway or Vercel

Coming Next

- Admin authentication (JWT, roles)
- Product catalog CRUD + bulk CSV import
- Vendor management, customer accounts, cart & checkout
- Razorpay payments, CRM (leads, pipeline, email campaigns), orders

Summary

The backend foundation is complete. The platform is ready to build business features on top of this base.